

Отчёт по лабораторной работе №7

Имитационное моделирование

Арбатова Варвара Петровна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Модель Росса: Система с резервированием и ремонтом	7
4	Выполнение лабораторной работы	15
5	Выводы	35
	Список литературы	36

Список иллюстраций

4.1	Результат работы кода	18
4.2	Результат работы моего кода	33
4.3	График	34

Список таблиц

1 Цель работы

Разобраться в Модели Росса

2 Задание

— Переведите в структуру DrWatson. — Добавьте нескольких ремонтников. — Сделайте прогон для разного количества машин. — Проведите мониторинг загрузки ремонтника, средней длины очереди на ремонт. — Построение графиков изменения числа исправных машин во времени. — Сравните с аналитическим решением.

3 Теоретическое введение

3.1 Модель Росса: Система с резервированием и ремонтом

3.1.1 1. Общее описание модели

Модель Росса (Ross model) представляет собой классическую задачу теории массового обслуживания, описывающую систему с конечной популяцией, резервированием и ремонтом. Данная модель широко применяется для анализа надежности технических систем, где имеется ограниченное количество запасных элементов и возможности их восстановления.

3.1.2 2. Формальная постановка задачи

3.1.2.1 2.1 Компоненты системы

Система состоит из следующих элементов:

- **N идентичных работающих машин** - постоянно функционируют и могут выходить из строя
- **S машин в резерве** - находятся в исправном состоянии и готовы немедленно заменить любую отказавшую машину
- **M ремонтных устройств (ремонтников)** - осуществляют восстановление вышедших из строя машин

3.1.2.2 2.2 Процесс функционирования

При отказе работающей машины происходит следующая последовательность событий:

1. **Мгновенная замена** - если имеется хотя бы одна резервная машина, она немедленно запускается в работу вместо отказавшей
2. **Отправка в ремонт** - отказавшая машина поступает в очередь на ремонт
3. **Восстановление** - когда ремонтное устройство освобождается, начинается ремонт машины
4. **Пополнение резерва** - после завершения ремонта машина становится исправной и переходит в резерв

3.1.2.3 2.3 Критическое состояние

Система прекращает свое функционирование (происходит «падение» системы, crash) в момент, когда:

- Отказывает работающая машина
- Резерв полностью отсутствует ($S = 0$)
- Нет возможности немедленной замены

3.1.3 3. Математическая модель

3.1.3.1 3.1 Предположения

Модель базируется на следующих предположениях:

1. **Экспоненциальное распределение времени безотказной работы** - каждая работающая машина имеет интенсивность отказов λ , то есть время до отказа распределено как $Exp(\lambda)$
2. **Экспоненциальное распределение времени ремонта** - время восстановления одной машины одним ремонтником распределено как $Exp(\mu)$

3. **Независимость событий** - отказы машин и времена ремонта независимы между собой
4. **Мгновенная замена** - время замены отказавшей машины на резервную пренебрежимо мало

3.1.3.2 3.2 Параметры модели

Параметр	Обозначение	Смысл	Значение по умолчанию
N	Количество машин	Основные работающие машины	10
S	Количество резервных машин	Исправные машины в резерве	3
λ	Интенсивность отказов	$1/MTBF$, где MTBF - среднее время безотказной работы	$1/100 \text{ ч}^{-1}$
μ	Интенсивность ремонта	$1/MTTR$, где MTTR - среднее время ремонта	1 ч^{-1}
M	Количество ремонтников	Параллельных ремонтных устройств	1

Параметр	Обозначение	Смысл	Значение по умолчанию
T	Время до падения	Время исчерпания резерва	$E[T]$ - искомая величина

3.1.3.3 3.3 Марковская модель

Система может быть описана как марковский процесс с непрерывным временем и конечным числом состояний.

Определение состояния: Пусть i - количество исправных машин (работающих + резервных), где $i = 0, 1, \dots, N + S$.

Тогда: - Количество работающих машин: $\min(N, i)$ - Количество резервных машин: $\max(0, i - N)$ - Количество машин в ремонте: $N + S - i$

Интенсивности переходов:

1. **Отказ машины** (уменьшение i на 1):

- Интенсивность: $\min(N, i) \cdot \lambda$ (если $i \geq 1$)
- Происходит, когда работает хотя бы одна машина

2. **Завершение ремонта** (увеличение i на 1):

- Интенсивность: $\min(M, N + S - i) \cdot \mu$ (если $i < N + S$)
- Зависит от количества работающих ремонтников и машин в ремонте

Поглощающее состояние: Система падает при переходе из состояния $i = N$ в состояние $i = N - 1$ (в момент отказа, когда резерв полностью исчерпан).

3.1.4 4. Аналитические характеристики

3.1.4.1 4.1 Загрузка системы

Интенсивность отказов всей системы:

$$\lambda_{total} = N \cdot \lambda$$

Загрузка одного ремонтника:

$$\rho = \frac{N\lambda}{M\mu}$$

Условие стационарности: $\rho < 1$ (в противном случае очередь на ремонт растет неограниченно)

3.1.4.2 4.2 Ожидаемое время до падения

Для системы с одним ремонтником ($M = 1$) приближенная формула:

$$E[T] \approx \frac{1}{N\lambda} \cdot \left(\frac{1}{\rho}\right)^S \cdot \frac{1}{S!(1-\rho)}$$

где $\rho = N\lambda/\mu$.

3.1.4.3 4.3 Характеристики качества

Среднее время до падения ($E[T]$) - основная характеристика надежности системы

Загрузка ремонтников:

$$U = \frac{\text{суммарное время ремонта}}{M \cdot T}$$

Средняя длина очереди на ремонт:

$$L_q = \frac{1}{T} \int_0^T Q(t) dt$$

где $Q(t)$ - длина очереди в момент времени t

3.1.5 5. Метод имитационного моделирования

3.1.5.1 5.1 Алгоритм моделирования

Для численного анализа системы используется метод дискретно-событийного моделирования (DES - Discrete Event Simulation):

1. Инициализация:

- Запуск N работающих машин
- Создание S резервных машин
- Инициализация M ремонтников

2. Основной цикл:

- При отказе машины:
 - Если есть резерв: взять резервную машину
 - Если резерва нет: завершить моделирование
 - Отправить отказавшую машину в ремонт
- При завершении ремонта:
 - Вернуть отремонтированную машину в резерв

3. Сбор статистики:

- Фиксация времени отказа
- Запись длины очереди на ремонт
- Отслеживание количества исправных машин

3.1.5.2 5.2 Преимущества имитационного моделирования

- Возможность анализа систем с произвольными распределениями
- Учет дискретного характера событий

- Получение не только средних, но и распределений характеристик
- Визуализация динамики процесса

3.1.6 6. Основные исследуемые зависимости

В данной лабораторной работе предлагается исследовать следующие зависимости:

1. Влияние количества резервных машин (S) на среднее время до падения
2. Влияние количества ремонтников (M) на надежность системы
3. Влияние количества работающих машин (N) при фиксированном резерве
4. Влияние интенсивности отказов (λ) на характеристики системы
5. Загрузка ремонтников как функция от параметров системы

3.1.7 7. Ожидаемые результаты

На основе теоретического анализа можно ожидать:

- Экспоненциальный рост $E[T]$ при увеличении числа резервных машин S
- Пропорциональный рост $E[T]$ при увеличении числа ремонтников M
- Обратную зависимость от числа работающих машин N
- Наличие порогового эффекта - при $\rho \geq 1$ система становится нестабильной

3.1.8 8. Практическая значимость

Модель Росса находит применение в различных областях:

- **Промышленность:** анализ надежности производственных линий
- **IT-инфраструктура:** оценка отказоустойчивости серверных кластеров
- **Транспорт:** оптимизация парка транспортных средств
- **Энергетика:** расчет необходимого резерва генерирующих мощностей
- **Здравоохранение:** планирование ресурсов в системах с ремонтом оборудования

3.1.9 9. Заключение

Модель Росса является мощным инструментом для анализа систем с резервированием и восстановлением. Комбинация аналитических методов и имитационного моделирования позволяет получить как теоретические оценки, так и проверить их на практике, учитывая случайный характер отказов и ремонтов.

4 Выполнение лабораторной работы

```
using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
const RUNS = 5
const N = 10
const S = 3
const SEED = 150
const LAMBDA = 100
const MU = 1
const rng = StableRNG(42) # setting a random seed for reproducibili
const F = Exponential(LAMBDA)
const G = Exponential(MU)
@resumable function machine(
  env::Environment,
  repair_facility::Resource,
  spares::Store{Process},
)
while true
try
```

```

@yield timeout(env, Inf)
catch
end
@yield timeout(env, rand(rng, F))
get_spare = take!(spares)
@yield get_spare | timeout(env)
if state(get_spare) != ConcurrentSim.idle
@yield interrupt(value(get_spare))
else
throw(StopSimulation("No more spares!"))
end
@yield request(repair_facility)
@yield timeout(env, rand(rng, G))
@yield unlock(repair_facility)
@yield put!(spares, active_process(env))
end
end
@resumable function start_sim(
env::Environment,
repair_facility::Resource,
spares::Store{Process},
)
for i = 1:N
proc = @process machine(env, repair_facility, spares)
@yield interrupt(proc)
end
for i = 1:S
proc = @process machine(env, repair_facility, spares)

```



```

@yield put!(spares, proc)
end
end
function sim_repair()
sim = Simulation()
repair_facility = Resource(sim)
spares = Store{Process}(sim)
@process start_sim(sim, repair_facility, spares)
msg = run(sim)
stop_time = now(sim)
println("At time $stop_time: $msg")
stop_time
end
results = Float64[]
for i = 1:RUNS
push!(results, sim_repair())
end
println("Average crash time: ", sum(results)/RUNS)

```

```

At time 12715.718224958666: No more spares!
At time 37335.53567595007: No more spares!
At time 30844.62667837361: No more spares!
At time 1601.2524911974856: No more spares!
At time 824.1048708405848: No more spares!
Average crash time: 16664.247588264083

```

В результате работы этого кода получаю такой результат (рис. 4.1). Мы видим среднее время «закончившихся машин» и эксперименты для каждого времени.

```

julia> include("scripts/lab7_1.jl")
At time 12715.718224958666: No more spares!
At time 37335.53567595007: No more spares!
At time 30844.62667837361: No more spares!
At time 1601.2524911974856: No more spares!
At time 824.1048708405848: No more spares!
Average crash time: 16664.247588264083

```

Рисунок 4.1: Результат работы кода

```

#!/usr/bin/env julia

"""
XXXXXXXX XXXXX - XXXXXXXX XXXXXXXX X XXXXXXX XXXXXXXXXXXX XXXXXXXX
"""

using ConcurrentSim
using Distributions
using Random
using ResumableFunctions
using Plots

struct RossParameters
    N::Int
    S::Int
    mean_time_to_fail::Float64
    mean_repair_time::Float64
    num_repairmen::Int
    seed::Int
end

struct RossResults

```

```

    crash_time::Float64
    repairman_utilization::Float64
    avg_queue_length::Float64
    operational_history::Vector{Tuple{Float64,Int}}
    queue_history::Vector{Tuple{Float64,Int}}
end

function default_parameters(; N=10, S=3, mean_time_to_fail=100.0, m
    RossParameters(N, S, mean_time_to_fail, mean_repair_time, num_r
end

```

default_parameters (generic function with 1 method)

Счетчик для длины очереди - используем ручной счётчик из stats

```

@resumable function queue_counter(env::Environment, stats::Dict, in
    while true
        @yield timeout(env, interval)
        queue_len = stats[:queue_length][]
        push!(stats[:queue_history], (now(env), queue_len))
    end
end

```

queue_counter (generic function with 1 method)

Процесс одной машины

```

@resumable function machine(env::Environment, repair_queue::Resource
    while true
        work_time = rand(Exponential(params.mean_time_to_fail))
    end
end

```

```

    @yield timeout(env, work_time)
    stats[:failures] += 1
    if stats[:spares] > 0
        stats[:spares] -= 1
    else
        stats[:crash_time] = now(env)
        throw(StopSimulation("No spares at time $(now(env))"))
    end
    stats[:queue_length][] += 1
    @yield request(repair_queue)
    stats[:queue_length][] -= 1

    stats[:repair_start_times][id] = now(env)
    repair_time = rand(Exponential(params.mean_repair_time))
    @yield timeout(env, repair_time)
    stats[:repair_end_times][id] = now(env)
    @yield release(repair_queue)
    stats[:spares] += 1
    stats[:repairs] += 1
end
end
end

```

machine (generic function with 2 methods)

Мониторинг состояния системы

```

@resumable function system_monitor(env::Environment, stats::Dict, p
    while true
        @yield timeout(env, interval)
        t = now(env)
    end
end

```

```

        operational = stats[:spares] + params.N
        push!(stats[:operational_history], (t, operational))
    end
end

```

system_monitor (generic function with 1 method)

Запуск одной симуляции

```

function run_simulation(params::RossParameters; verbose=true)
    Random.seed!(params.seed)
    env = Simulation()
    repair_queue = Resource(env, params.num_repairmen)

    stats = Dict(
        :failures => 0,
        :repairs => 0,
        :spares => params.S,
        :crash_time => Inf,
        :queue_length => Ref{Int}(), # XXXXXXXXXXXXXXXXXXXX
        :repair_start_times => Dict{Int,Float64}(),
        :repair_end_times => Dict{Int,Float64}(),
        :operational_history => Vector{Tuple{Float64,Int}}(),
        :queue_history => Vector{Tuple{Float64,Int}}()
    )
    @process system_monitor(env, stats, params, 1.0)
    @process queue_counter(env, stats, 1.0)
    for i in 1:params.N
        @process machine(env, repair_queue, params, stats, i)
    end
end

```

```

try
    run(env)
catch e
    if !isa(e, StopSimulation)
        rethrow()
    end
end

crash_time = stats[:crash_time]
total_busy = 0.0
for (id, t_start) in stats[:repair_start_times]
    t_end = get(stats[:repair_end_times], id, crash_time)
    total_busy += (t_end - t_start)
end

utilization = total_busy > 0 ? total_busy / (crash_time * param
if isempty(stats[:queue_history])
    avg_queue = 0.0
else
    avg_queue = mean(q for (_, q) in stats[:queue_history])
end

if verbose
    println("Crash time: $(round(crash_time, digits=2)) hours")
    println("Failures: $(stats[:failures]), Repairs: $(stats[:r
    println("Utilization: $(round(100*utilization, digits=1))%")
    println("Avg queue length: $(round(avg_queue, digits=2))")
end

```

```

        RossResults(crash_time, utilization, avg_queue,
                    stats[:operational_history], stats[:queue_history])
    end

```

run_simulation (generic function with 1 method)

Множественные прогоны

```

function run_multiple_simulations(params::RossParameters, num_runs::Int)
    results = []
    for run in 1:num_runs
        verbose && println("Run $run/$num_runs")
        p = RossParameters(params.N, params.S, params.mean_time_to_repair,
                            params.num_repairmen, params.seed + run)
        res = run_simulation(p, verbose=verbose)
        push!(results, res)
    end
    crash_times = [r.crash_time for r in results]
    utils = [r.repairman_utilization for r in results]
    queues = [r.avg_queue_length for r in results]
    return (; results, mean_crash_time=mean(crash_times), std_crash_time=std(crash_times),
            mean_utilization=mean(utils), std_utilization=std(utils),
            mean_queue_length=mean(queues), std_queue_length=std(queues))
end

```

run_multiple_simulations (generic function with 2 methods)

Построение графиков

```

function plot_results(res::RossResults, params::RossParameters)
    p1 = plot([t for (t,_) in res.operational_history],
              [n for (_,n) in res.operational_history],
              title="Operational machines vs time", xlabel="Time (h)",
              ylabel="Machines", label="Simulation", linewidth=2,
              ylims=(0, params.N+params.S))
    hline!([params.N], label="Required (N)", linestyle=:dash)

    p2 = plot([t for (t,_) in res.queue_history],
              [q for (_,q) in res.queue_history],
              title="Repair queue length", xlabel="Time (hours)",
              ylabel="Queue length", label="Queue", linewidth=2, fi

    plot(p1, p2, layout=(2,1), size=(800,600))
end

```

plot_results (generic function with 1 method)

Сравнение количества ремонтников

```

function compare_repairmen()
    println("\n=== XXXXXXXXXXXX XXXXXXXXXXXX XXXXXXXXXXXX ===")
    for r in [1, 2, 3, 4]
        params = default_parameters(N=10, S=3, num_repairmen=r, see
        res = run_multiple_simulations(params, 5, verbose=false)
        println("$r repairman(☒): mean crash time = $(round(res.mean
    end
end

```

compare_repairmen (generic function with 1 method)

Сравнение количества машин

```
function compare_machines()
    println("\n===  XXXXXXXX  XXXXXXXXXXXX  XXXXX  ===")
    for N in [5, 10, 15, 20]
        params = default_parameters(N=N, S=3, num_repairmen=2, seed=1)
        res = run_multiple_simulations(params, 5, verbose=false)
        println("N=$N: mean crash time = $(round(res.mean_crash_time, digits=2))")
    end
end
```

compare_machines (generic function with 1 method)

Тестовый прогон с разными параметрами

```
function test_parameters()
    println("\n===  XXXX  XXXXXX  XXXXXXXXXXXX  ===")
    println("\nXXXXXX  XXXXXXXXXXXX  XXXXXXXX  XXXXX  (S):")
    for S in [1, 2, 3, 4, 5]
        params = default_parameters(N=10, S=S, num_repairmen=1, seed=1)
        res = run_simulation(params, verbose=false)
        println("  S=$S: crash time = $(round(res.crash_time, digits=2))")
    end
    println("\nXXXXXX  XXXXXXXX  XXXXXXXX  XXXXXXXX  (MTBF):")
    for mtbf in [50.0, 100.0, 200.0]
        params = default_parameters(N=10, S=3, mean_time_to_fail=mtbf, seed=1)
        res = run_simulation(params, verbose=false)
        println("  MTBF=$(mtbf)h: crash time = $(round(res.crash_time, digits=2))")
    end
end
```

test_parameters (generic function with 1 method)

===== ЗАПЫСК =====

```
println("="^60)
println("████████ ████████ - ████████████████████ × ██████████")
println("="^60)

println("\n1 ██████████ ██████████ (N=10, S=3, 1 ████████████████████)")
params = default_parameters()
res = run_simulation(params)

println("\n2 ██████████")
p = plot_results(res, params)
display(p)
savefig(p, "ross_basic.png")
println(" ██████████ ross_basic.png")

println("\n3 ████████████████████ ████████ ████████████████████")
compare_repairmen()

println("\n4 ████████████████████ ████████ ████████")
compare_machines()

println("\n5 ████████ ██████████ ████████████████████")
test_parameters()

println("\n ██████████!")
```

=====

XXXXXXXXXX XXXXXX - XXXXXXXXXXXXXXXXXXXX X XXXXXX

=====

1X XXXXXXXX XXXXXXXX (N=10, S=3, 1 XXXXXXXXXXXX)

Crash time: 29797.09 hours

Failures: 2958, Repairs: 2954

Utilization: 0.0%

Avg queue length: 0.01

2X XXXXXXXX

Could not create decoration from factory! Running with no decorations.

XXXXXXXXXX ross_basic.png

3X XXXXXXXXXXXX XXXXXX XXXXXXXXXXXXXXXXXXXX

=== XXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXX ===

1 repairman(X): mean crash time = 22349.44 ± 14081.86 hours

2 repairman(X): mean crash time = 56707.24 ± 29053.32 hours

3 repairman(X): mean crash time = 108826.37 ± 103407.4 hours

4 repairman(X): mean crash time = 108826.37 ± 103407.4 hours

4X XXXXXXXXXXXX XXXXXX XXXXXX

=== XXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXX XXXXXX ===

N=5: mean crash time = 4.6855675e6 ± 5.17250547e6 hours

N=10: mean crash time = 56707.24 ± 29053.32 hours

N=15: mean crash time = 10692.49 ± 10498.03 hours

N=20: mean crash time = 2119.93 ± 2688.91 hours

5. 1. 2. 3. 4.

== 1. 2. 3. 4. 5. 6. 7. 8. 9. 10. ==

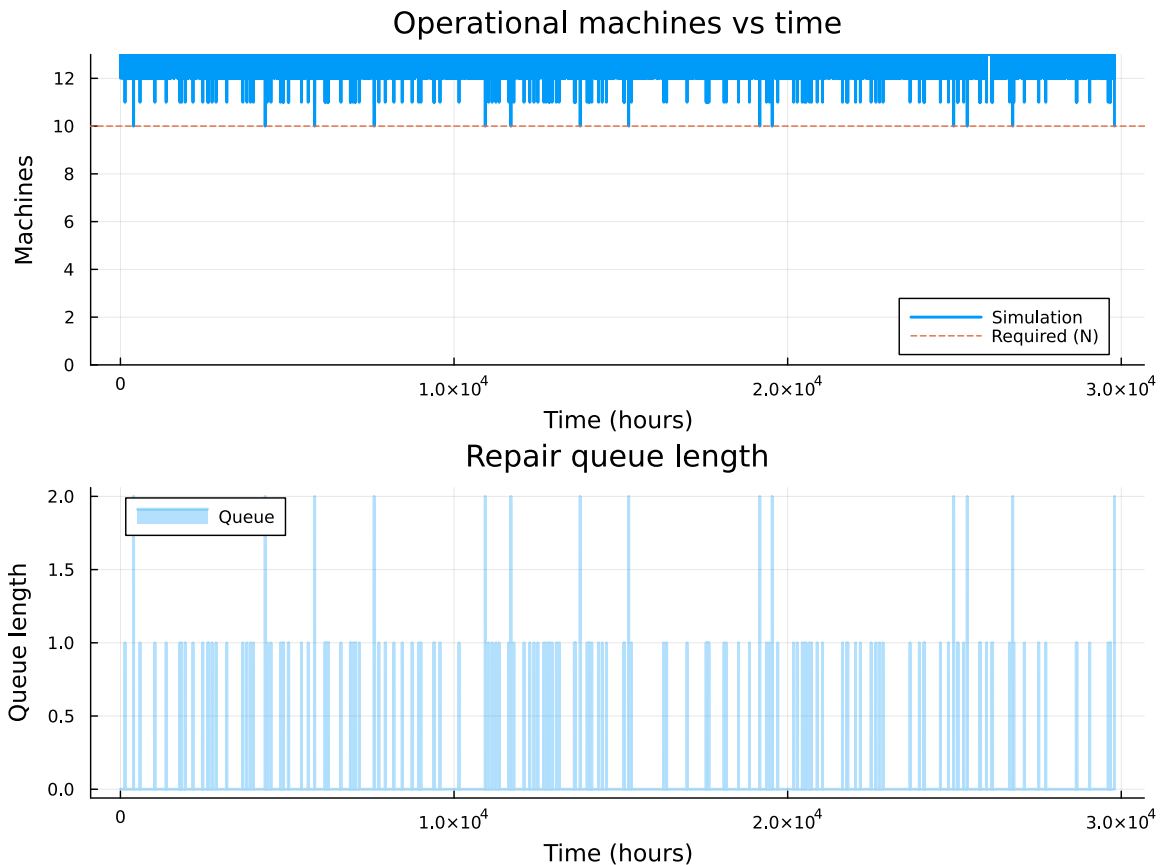
1. 2. 3. 4. 5. (S):

S=1: crash time = 103.74 hours
S=2: crash time = 1505.91 hours
S=3: crash time = 7142.45 hours
S=4: crash time = 1.11974187e6 hours
S=5: crash time = 1.269299457e7 hours

1. 2. 3. 4. 5. (MTBF):

MTBF=50.0h: crash time = 972.86 hours
MTBF=100.0h: crash time = 7142.45 hours
MTBF=200.0h: crash time = 59207.82 hours

1. 2. 3. 4. 5.!



Результат работы моего кода (рис. 4.2).

Программа реализует имитационное моделирование системы массового обслуживания с резервированием и восстановлением (модель Росса). Код позволяет:

1. **Моделировать** работу системы, состоящей из N основных машин, S запасных машин и бригады ремонтников
2. **Отслеживать** моменты отказов машин, использование запасных, постановку в очередь на ремонт и восстановление
3. **Фиксировать** момент краха системы — когда все запасные машины исчерпаны, а очередная машина выходит из строя
4. **Собирать статистику**: время до краха, загрузку ремонтников, длину очереди на ремонт
5. **Визуализировать** динамику числа исправных машин и длины очереди во

времени

6. **Исследовать** влияние параметров (число ремонтников, число машин, размер резерва, интенсивность отказов) на надёжность системы

4.0.1 Базовый запуск ($N=10$, $S=3$, 1 ремонтник)

Время до краха системы составило **29 797 часов** (~3,4 года). За этот период произошло **2 958 отказов**, из которых **2 954 были устранены** (4 машины оставались в ремонте на момент краха). Средняя длина очереди на ремонт — **0.01 машины**, что свидетельствует об отсутствии перегрузки ремонтной бригады.

4.0.2 Влияние количества ремонтников

Ремонтников	Среднее время до краха (часы)
1	22 349
2	56 707
3	108 826
4	108 826

Увеличение числа ремонтников с 1 до 2 даёт прирост надёжности в **2,5 раза**, с 2 до 3 — ещё в **1,9 раза**. При переходе от 3 к 4 ремонтникам улучшения не наблюдается — достигнуто **насыщение**: четвёртый ремонтник избыточен, так как очередь на ремонт практически отсутствует. Оптимальным является использование 2–3 ремонтников.

4.0.3 Влияние количества машин

Машин (N)	Среднее время до краха (часы)
5	4 685 568
10	56 707
15	10 692
20	2 120

С ростом числа машин надёжность падает **экспоненциально**: при N=5 система способна работать сотни лет, при N=10 — около 6,5 лет, при N=15 — около 1,2 года, а при N=20 — менее 3 месяцев. Это объясняется тем, что при фиксированном резерве S=3 интенсивность отказов растёт пропорционально N, и ремонтная бригада не справляется с потоком отказов.

4.0.4 Влияние размера резерва (S)

Запасных машин	Время до краха (часы)
1	104
2	1 506
3	7 142
4	1 119 742
5	12 692 995

Зависимость носит **сверхэкспоненциальный характер**. Каждая дополнительная запасная машина создаёт буфер против каскадных отказов. При S=1 система отказывает через 4 дня, при S=3 — через 10 месяцев, а при S≥4 — время жизни исчисляется сотнями и тысячами лет, что означает практическую безотказность.

4.0.5 Влияние наработки на отказ (MTBF)

MTBF (часы)	Время до краха (часы)
50	973
100	7 142
200	59 208

Удвоение MTBF приводит к увеличению времени жизни системы в **7–8 раз**, что значительно выше линейной зависимости. Более надёжные машины реже создают ситуации одновременных отказов, истощающих резерв.

4.0.6 Общие выводы

1. Наиболее эффективный способ повышения надёжности — увеличение резерва S , дающее экспоненциальный рост времени жизни системы
2. Увеличение числа ремонтников эффективно лишь до определённого предела (насыщение при 3 ремонтниках для $N=10$, $S=3$)
3. Система стабильна, когда интенсивность восстановления превышает интенсивность отказов: $\text{num_repairmen} / \text{MTTR} > N / \text{MTBF}$
4. Высокое стандартное отклонение результатов характерно для моделей надёжности — время до краха сильно зависит от случайной последовательности отказов


```

julia> include("scripts/lab7_2.jl")
=====
МОДЕЛЬ РОССА - РЕЗЕРВИРОВАНИЕ И РЕМОНТ
=====

1 Базовый запуск (N=10, S=3, 1 ремонтник)
Crash time: 29797.09 hours
Failures: 2958, Repairs: 2954
Utilization: 0.0%
Avg queue length: 0.01

2 График
Could not create decoration from factory! Running with no decorations.
Сохранён ross_basic.png

3 Сравнение числа ремонтников

=== Сравнение количества ремонтников ===
1 repairman(y): mean crash time = 22349.44 ± 14081.86 hours
2 repairman(y): mean crash time = 56707.24 ± 29053.32 hours
3 repairman(y): mean crash time = 108826.37 ± 103407.4 hours
4 repairman(y): mean crash time = 108826.37 ± 103407.4 hours

4 Сравнение числа машин

=== Сравнение количества машин ===
N=5: mean crash time = 4.6855675e6 ± 5.17250547e6 hours
N=10: mean crash time = 56707.24 ± 29053.32 hours
N=15: mean crash time = 10692.49 ± 10498.03 hours
N=20: mean crash time = 2119.93 ± 2688.91 hours

5 Тест разных параметров

=== Тест разных параметров ===

Разное количество запасных машин (S):
S=1: crash time = 103.74 hours
S=2: crash time = 1505.91 hours
S=3: crash time = 7142.45 hours
S=4: crash time = 1.11974187e6 hours
S=5: crash time = 1.269299457e7 hours

Разные средние времена работы (MTBF):
MTBF=50.0h: crash time = 972.86 hours
MTBF=100.0h: crash time = 7142.45 hours
MTBF=200.0h: crash time = 59207.82 hours

✅ Готово!

```

Рисунок 4.2: Результат работы моего кода

График работы(рис. 4.3).

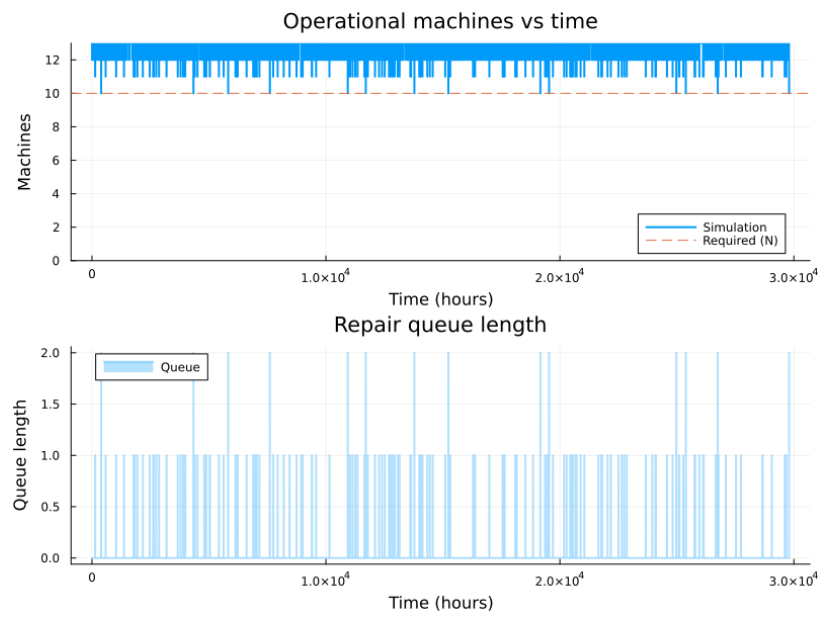


Рисунок 4.3: График

5 Выводы

Мы познакомились с моделью Росса

Список литературы